CMC SportBooking - Guide Technique Complet (Laravel API + React)

1. Objectif du projet

CMC SportBooking est une application full-stack pour gerer:

- authentification multi-portail (`admin` , `stagiaire` , `monitor`)
- gestion des terrains
- gestion des reservations
- gestion des stagiaires
- gestion des moniteurs (monitoring)
- verification email et recuperation mot de passe par code

Architecture:

- Backend API: `backend/` (Laravel)
- Frontend SPA: `frontend/` (React + Redux Toolkit + Vite)

---

2. Stack et structure

2.1 Backend Laravel

Fichiers clés:

- `backend/routes/api.php`
- `backend/bootstrap/app.php`
- `backend/app/Http/Middleware/AdminOnly.php`
- `backend/app/Http/Controllers/Api/AuthController.php`
- `backend/app/Http/Controllers/Api/ReservationController.php`
- `backend/app/Http/Controllers/Api/StagiaireController.php`
- `backend/app/Http/Controllers/Api/MonitorController.php`
- `backend/database/migrations/2026_03_06_000014_create_email_verification_codes_table.php`

2.2 Frontend React

Fichiers clés:

- `frontend/src/App.jsx`
- `frontend/src/api/client.js`
- `frontend/src/components/ProtectedRoute.jsx`
- `frontend/src/components/ShellLayout.jsx`
- `frontend/src/features/auth/authSlice.js`
- `frontend/src/features/reservations/reservationsSlice.js`
- `frontend/src/features/monitors/monitorsSlice.js`
- `frontend/src/pages/LoginPage.jsx`
- `frontend/src/pages/RegisterPage.jsx`
- `frontend/src/pages/ForgotPasswordPage.jsx`
- `frontend/src/pages/ReservationsPage.jsx`
- `frontend/src/pages/ReservationFormPage.jsx`

- `frontend/src/pages/MonitorsPage.jsx`
- `frontend/src/pages/TerrainFormPage.jsx`
- `frontend/src/pages/TerrainsPage.jsx`
- `frontend/src/i18n.js`

---

3. API Laravel: configuration et sécurité

3.1 Routing API

Dans `backend/routes/api.php`, les routes sont separées en 2 blocs:

- Public:
  - `POST /login`
  - `POST /register`
  - `POST /email/verify-code`
  - `POST /email/resend-verification`
  - `POST /password/forgot`
  - `POST /password/verify-code`
  - `POST /password/reset`
- Protege (`auth.api`):
  - profil utilisateur (`/me` , `/logout` , `/me/password`)
  - CRUD terrains/matches/reservations/stagiaires/monitors
  - support messages

3.2 Middlewares

Dans `backend/bootstrap/app.php` :

- alias `auth.api` -> `ApiTokenAuth`
- alias `admin.only` -> `AdminOnly`

Dans `backend/app/Http/Middleware/AdminOnly.php` :

- bloque toute requete si user absent ou role different de `admin`

3.3 Auth par token

Flow:

1. Login valide email/password + portal
2. Si OK -> generation token brut `bin2hex(random_bytes(40))`
3. Hash du token en DB (`api_token`)
4. Front stocke token en `localStorage`
5. Axios envoie `Authorization: Bearer <token>` sur chaque requete

---

4. AuthController: toutes les fonctions

Fichier: `backend/app/Http/Controllers/Api/AuthController.php`

4.1 `register()`

- Valide champs inscription stagiaire
- Gmail obligatoire (`@gmail.com`)
- CIN/email uniques
- Cree user role `stagiaire`
- Genere `student_id` auto (`STG000001...`)
- Genere code verification email
- Envoie mail HTML verification

4.2 `verifyEmailCode()`

- Valide email + code 6 digits
- Verifie expiration du code
- Compte tentatives (max 5)
- Compare hash sha256
- Marque `email_verified_at`

4.3 `resendEmailVerificationCode()`

- Reenvoi code verification
- Cooldown anti-spam (~45 sec)

4.4 `forgotPassword()`

- Verifie existence email
- Genere code reset 6 chiffres
- Stocke hash code + expiration
- Envoie email reset

4.5 `verifyResetCode()`

- Valide code reset
- Gere expiration + tentatives
- Si code correct -> genere `reset_token`

4.6 `resetPassword()`

- Valide `reset_token`
- Change mot de passe
- Supprime session reset

4.7 `login()`

- Valide `portal in: admin, stagiaire, monitor`
- Refuse si email non verifie
- Controle role par portal:
  - `admin` -> role admin + email admin
  - `stagiaire` -> role stagiaire
  - `monitor` -> role monitor
- Retourne `token` + `user`

4.8 `me()` , `updateMe()` , `logout()` , `updatePassword()`

- `me()` : retourne user courant

- `updateMe()` : update profil (email/cin/classe/filiere...)
- `logout()` : annule token
- `updatePassword()` : change password avec ancien mdp

4.9 helpers internes

- `issueEmailVerificationCode()`
- `sendEmailVerificationCode()`

---

5. ReservationController: logique metier

Fichier: `backend/app/Http/Controllers/Api/ReservationController.php`

5.1 `index()`

- Charge reservations parent (`whereNull(parent_reservation_id)`)
- Si stagiaire -> seulement ses reservations
- Synchronise status depuis matches (`syncStatusesFromMatches`)

5.2 `store()`

- Cree reservation principale
- Gere mode join via `join_reservation_code`
- Regles metier:
  - pas dans le passe
  - duree max 2h
  - pas plus d une reservation/jour pour stagiaire
  - pas overlap terrain
  - pas overlap joueur (email/cin/nom)
- Genere `reservation_code` unique

5.3 `findByCode()`

- Recherche reservation principale par `reservation_code`
- Retourne details + liste joueurs (parent + enfants)

5.4 `update()`

- Update reservation admin
- Recontrole toutes regles overlap/duree/date
- Synchronise status avec match si necessaire

5.5 `updateStatus()`

- Endpoint monitor/admin: `PATCH /reservations/{reservation}/status`
- Status autorises: `completed` ou `cancelled`
- Met a jour reservation principale + joueurs associes

5.6 `addPlayer()`

- Ajoute stagiaire dans reservation via `student_id`
- Verifie conflits horaires et duplicates

5.7 Fonctions internes

- `generateReservationCode()`
- `hasSameDayReservationForUser()`
- `resolveStatusFromMatch()`
- `syncStatusesFromMatches()`

---

6. StagiaireController

Fichier: `backend/app/Http/Controllers/Api/StagiaireController.php`

Fonctions:

- `index()` -> liste stagiaires + compteur reservations
- `store()` -> creation stagiaire (Gmail + CIN unique + `student_id` auto)
- `update()` -> modification stagiaire
- `destroy()` -> suppression stagiaire + reservations associees (transaction)
- `generateStudentId()` -> genere code `STGxxxxxx`

---

7. MonitorController

Fichier: `backend/app/Http/Controllers/Api/MonitorController.php`

Fonctions:

- `index()` -> liste monitors
- `store()` -> creation monitor
- `update()` -> modification monitor
- `destroy()` -> suppression monitor

Important:

- role fixe: `monitor`
- email Gmail obligatoire
- `email_verified_at = now()` auto (pour ne pas bloquer login monitor)

---

8. Migration email verification

Fichier: `backend/database/migrations/2026_03_06_000014_create_email_verification_codes_table.php`

Table `email_verification_codes` :

- `user_id` (unique FK users)
- `code_hash`
- `attempts`
- `expires_at`
- timestamps

Backfill inclus dans migration:

- users sans verification -> `email_verified_at = now()`

---

9. Frontend React: wiring principal

9.1 `App.jsx`

Role:

- declaration de toutes les routes
- applique theme/langue globalement
- recharge user via `meThunk()` si token existe

Routes login:

- `/login_stagiaire`
- `/login_admin`
- `/login_monitor`

Routes monitor:

- `/monitor/reservations`
- `/monitor/profile`

9.2 `api/client.js`

- axios baseURL `/api`
- intercepteur request ajoute automatiquement Bearer token

9.3 `ProtectedRoute.jsx`

- bloque acces sans token
- force redirection selon role

9.4 `ShellLayout.jsx`

- layout global dashboard
- sidebar/topbar
- navigation dynamique selon role (`admin` , `stagiaire` , `monitor`)
- gestion theme + langue + menu user
- `Profile` cache pour monitor (sur demande)

---

10. Redux Toolkit slices

10.1 `authSlice.js`

Thunks:

- `loginThunk`
- `meThunk`
- `logoutThunk`
- `updatePasswordThunk`
- `updateProfileThunk`

Etat:

- `token` , `user` , `loading` , `error`

10.2 `reservationsSlice.js`

Thunks:

- `fetchReservations`
- `createReservation`
- `updateReservation`
- `deleteReservation`
- `updateReservationStatus`
- `addReservationPlayer`

10.3 `monitorsSlice.js`

Thunks:

- `fetchMonitors`
- `createMonitor`
- `updateMonitor`
- `deleteMonitor`

Gestion erreurs:

- parse des erreurs backend pour afficher message utile en UI

---

11. Pages principales et leurs fonctions

11.1 `LoginPage.jsx`

Fonctions clés:

- selection portal forcee (`forcedPortal`) ou libre
- submit login
- redirection auto apres login selon role
- switch theme/langue

11.2 `RegisterPage.jsx`

Flow 2 etapes:

1. Creation compte stagiaire
2. Verification code Gmail

Fonctions:

- `submit()` inscription
- `verifyEmail()` verification code
- `resendVerificationCode()` renvoi code

11.3 `ForgotPasswordPage.jsx`

Flow 3 etapes:

1. Envoi code reset
2. Verification code
3. Nouveau mot de passe

Fonctions:

- `sendCode()`
- `verifyCode()`
- `resetPassword()`

11.4 `MonitorsPage.jsx`

CRUD complet + search monitor

Fonctions:

- `onSubmit()` create/update monitor
- `onEdit()` pre-remplissage form
- `onDelete()` suppression monitor
- `filteredMonitors` (useMemo)

Special case:

- si erreur `Unauthorized`, ferme form et retourne a vue table

11.5 `ReservationsPage.jsx`

- Vue admin complete (details, joueurs, add player)
- Bloc monitor special:
  - input ID/Code reservation
  - select status (`completed/cancelled`)
  - bouton confirmer
  - tableau status + modification inline

Fonctions clés:

- `runReservationLookup()` (admin)
- `submitStatusFromForm()` (monitor)
- `submitStatusFromRow()` (monitor)
- `loadPlayersByCode()`

11.6 `ReservationFormPage.jsx`

- Creation/modif reservation
- Slots 1h / 2h
- filtre terrains par type
- prevention overlap cote UI

11.7 `TerrainFormPage.jsx` et `TerrainsPage.jsx`

- CRUD terrain
- types pros:
  - `Football 11`
  - `Volley`
  - `Futsal`

    - `Basketball`
- compatibilite legacy `Gazon` / `Basket`

---

12. i18n et UI

Fichier: `frontend/src/i18n.js`

- 3 langues: `fr`, `en`, `ar`
- ajout de nouvelles cles pour:
    - monitor management
    - reservation checker/status update
    - monitor login
    - terrainType et autres labels

---

13. Flux API <-> React (explication simple)

Exemple: login

1. User remplit form React
2. React dispatch `loginThunk`
3. Thunk appelle `POST /api/login`
4. Laravel valide credentials + role portal
5. Laravel retourne token + user
6. React stocke token en localStorage
7. axios ajoute token sur requetes suivantes
8. pages protegees deviennent accessibles

Exemple: monitor change status

1. User monitor saisit code/id reservation
2. React retrouve reservation locale
3. React dispatch `updateReservationStatus`
4. API `PATCH /api/reservations/{id}/status`
5. Laravel met status parent + enfants
6. React refresh list via `fetchReservations`

---

14. Commandes pour executer le projet

Depuis racine projet:

Backend

```
cd backend
composer install
copy .env.example .env
php artisan key:generate
php artisan migrate
php artisan serve
```

Frontend

```
cd frontend
npm.cmd install
npm.cmd run dev
```

Pages login:

- `http://127.0.0.1:5173/login_admin`
- `http://127.0.0.1:5173/login_stagiaire`
- `http://127.0.0.1:5173/login_monitor`

---

15. Résumé final

Ce projet implemente une architecture propre API + SPA:

- securite role-based
- verification Gmail + reset password par code
- reservation engine avec regles metier anti-conflit
- espace admin complet
- espace monitor specialise sur statut reservations
- UI multilingue et theming

Le code est structure autour de:

- Controllers Laravel (metier API)
- Redux slices (etat + appels API)
- Pages React (UX/flow)
- Middlewares (auth/permissions)

Ce guide couvre la logique de zero jusqu au fonctionnement complet.